What Predicted the 2008 Housing Crash?

Bubble Deflation vs. Economic Fundamentals

Introduction

The 2008 financial crisis produced the largest decline in U.S. home prices since the Great Depression, but the damage was not evenly distributed. Nevada lost over 55% of its peak home value; North Dakota lost nothing. Was the crash simply a bubble deflation, or did pre-crisis economic fundamentals determine which states were resilient? We test this using state-level FRED data (house price index, unemployment, income, population) for all 50 states plus DC, with the peak-to-trough HPI decline as the outcome variable.

Beyond the regression analysis, we investigate LLMs as prediction tools for small-sample economic tasks. Claude Opus 4.6 achieves mean RMSE 4.34 across 20 cross-validation splits versus OLS's 8.88. However, a zero-data control (the model receives only state names, no features or training data) achieves RMSE 2.58, indicating the model has memorized crash outcomes from pre-training. This reframes the experiment as a case study in *prior-likelihood interaction*: providing four features that explain 58% of variance degrades predictions from a model whose pre-trained prior is already near-perfect. The result illustrates a general Bayesian principle (incomplete data can hurt when the prior is strong) and identifies the conditions under which the predictive primitive hypothesis could be tested cleanly.

Data wrangling

The raw data consists of 204 CSV files downloaded from FRED, four time series for each of the 50 states plus DC spanning 2000-2012. We read each series, extract summary statistics at key dates, and assemble them into a single cross-sectional DataFrame with one row per state. Full data construction code is available in the project repository.

Computing crash severity from the House Price Index

For each state, we find the peak HPI value in the 2000-2012 window, then find the minimum HPI after that peak date. The crash percentage is `(trough - peak) / peak * 100`. We also

compute pre-crisis appreciation as the percent change in HPI from Q1 2003 to Q1 2006, which measures how much the bubble inflated before it burst.

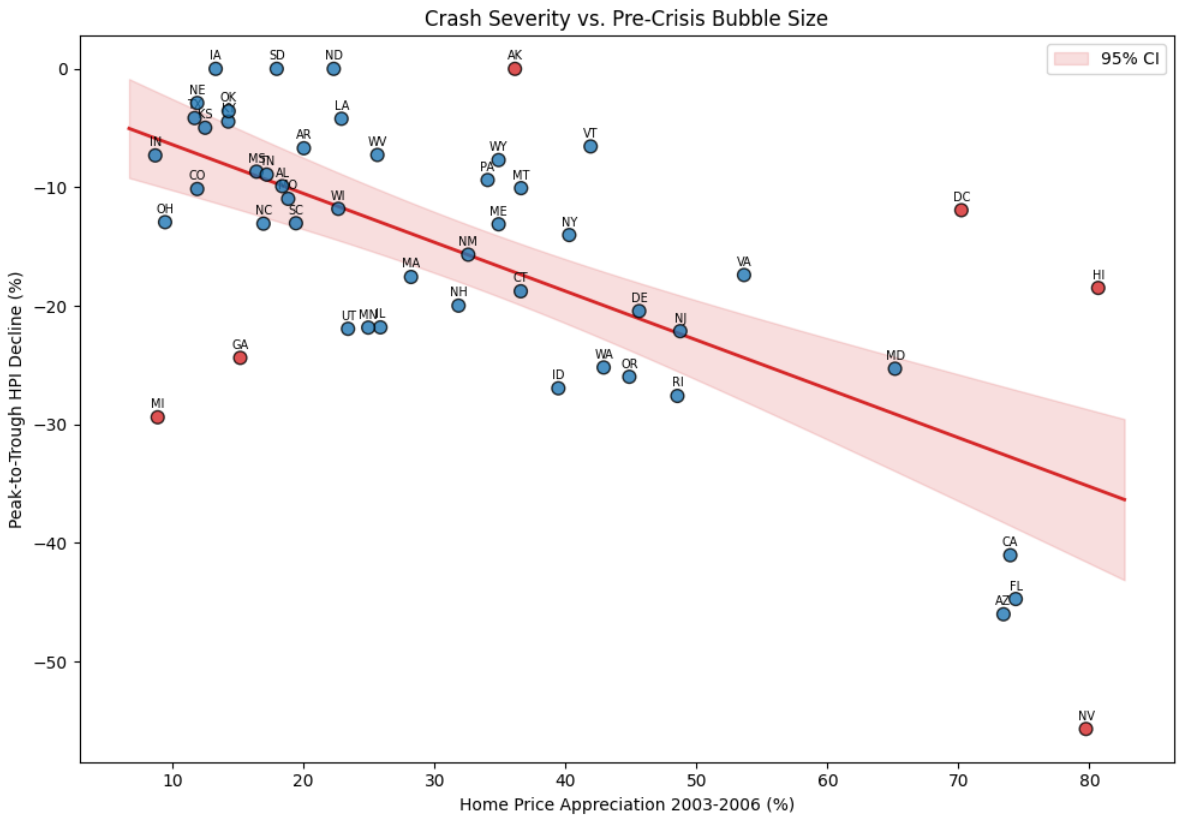
	state	appreciation_03_06	ur_2006	pcpi_2006	pop_growth_03_06	crash_pct
27	NV	79.738252	4.166667	39437	12.175467	-55.694418
2	AZ	73.441020	4.266667	34460	9.414569	-46.013732
8	FL	74.374694	3.283333	38009	6.838974	-44.729060
4	CA	73.961513	4.891667	41454	2.178650	-41.035641
21	MI	8.860281	6.866667	33563	-0.050502	-29.395725
38	RI	48.543015	5.258333	38719	-0.769689	-27.596454
11	ID	39.453794	3.391667	31244	7.722645	-26.951190
36	OR	44.873014	5.391667	34488	3.481644	-25.980296
19	MD	65.156794	3.850000	45293	2.385218	-25.299040
46	WA	42.914120	4.991667	39744	4.368168	-25.194308

Each row is one state. The outcome variable `crash_pct` is the peak-to-trough percentage decline in the FHFA House Price Index (2006-2012). More negative values indicate a more severe crash. The four predictors are pre-crisis indicators observable before the crash began:

- **appreciation_03_06**: Home price appreciation 2003-2006 (%). Proxy for bubble size.
- **ur_2006**: Unemployment rate in 2006 (%). Pre-crash labor market health.
- **pcpi_2006**: Per-capita personal income in 2006 ($). Economic baseline.
- **pop_growth_03_06**: Population growth 2003-2006 (%). Migration-driven demand.

The dataset contains 51 observations (50 states + DC). Crash severity ranges from 0% (no decline: ND, IA) to -55.7% (Nevada).

Visualizations



r = -0.684, R-squared = 0.467

Analysis: does the bubble explain the crash?

```
lm_simple = smf.ols('crash_pct ~ appreciation_03_06', data=df).fit()
print(f'R-squared: {lm_simple.rsquared:.3f}')
print(pd.DataFrame({'coef': lm_simple.params, 'std err': lm_simple.bse,
 ↪  'p-value': lm_simple.pvalues}).round(4))
```

```
R-squared: 0.467
                      coef  std err  p-value
Intercept          -2.2788   2.4212   0.3512
appreciation_03_06 -0.4120   0.0629   0.0000
```

```
lm_multi = smf.ols('crash_pct ~ appreciation_03_06 + ur_2006 + pcpi_2006 +
↪  pop_growth_03_06', data=df).fit()
print(f'R-squared: {lm_multi.rsquared:.3f}')
print(pd.DataFrame({'coef': lm_multi.params, 'std err': lm_multi.bse,
↪  'p-value': lm_multi.pvalues}).round(4))
```

```
R-squared: 0.582
                       coef  std err  p-value
Intercept           10.3390  10.5473   0.3321
appreciation_03_06  -0.3765   0.0751   0.0000
ur_2006             -2.2752   1.2133   0.0671
pcpi_2006            0.0000   0.0002   0.9490
pop_growth_03_06    -1.4420   0.4845   0.0046
```

```
from statsmodels.stats.anova import anova_lm
print(anova_lm(lm_simple, lm_multi))
```

```
   df_resid          ssr  df_diff     ss_diff         F    Pr(>F)
0      49.0  4012.335552      0.0         NaN       NaN       NaN
1      46.0  3151.414654      3.0  860.920898  4.188845  0.010558
```

Model 1 (appreciation only) explains about 47% of the variance in crash severity. Each additional percentage point of pre-crisis appreciation corresponds to roughly a 0.41 percentage point larger crash. Model 2 adds unemployment, income, and population growth, raising R-squared to about 0.58. The F-test above evaluates whether the three additional predictors are jointly significant, that is, whether Model 2 explains significantly more variance than Model 1. If the p-value is below 0.05, the economic fundamentals add predictive power beyond the bubble alone.

Simulation: is the relationship real?

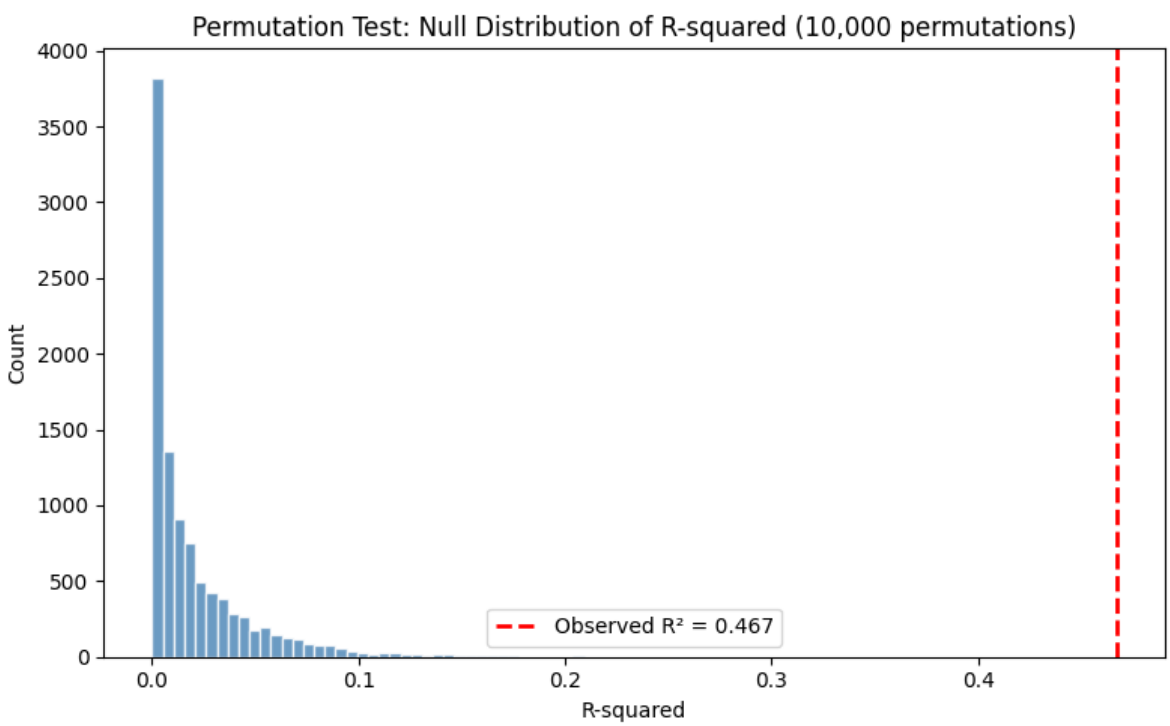
Under the null hypothesis that pre-crisis appreciation has no relationship with crash severity, we would expect any apparent correlation to vanish if we shuffled the crash labels. To test this, we permute `crash_pct` across states 10,000 times, refit the simple regression each time, and collect the null distribution of R-squared values. We then compare our observed R-squared to this distribution.

```
np.random.seed(42)
n_perm = 10_000
null_r2 = np.empty(n_perm)
```

```python
for i in range(n_perm):
    df_perm = df.copy()
    df_perm['crash_pct'] = np.random.permutation(df['crash_pct'].values)
    m = smf.ols('crash_pct ~ appreciation_03_06', data=df_perm).fit()
    null_r2[i] = m.rsquared

observed_r2 = lm_simple.rsquared
p_value = np.mean(null_r2 >= observed_r2)
print(f'Observed R-squared: {observed_r2:.4f}')
print(f'Permutation p-value: {p_value:.4f}')
```

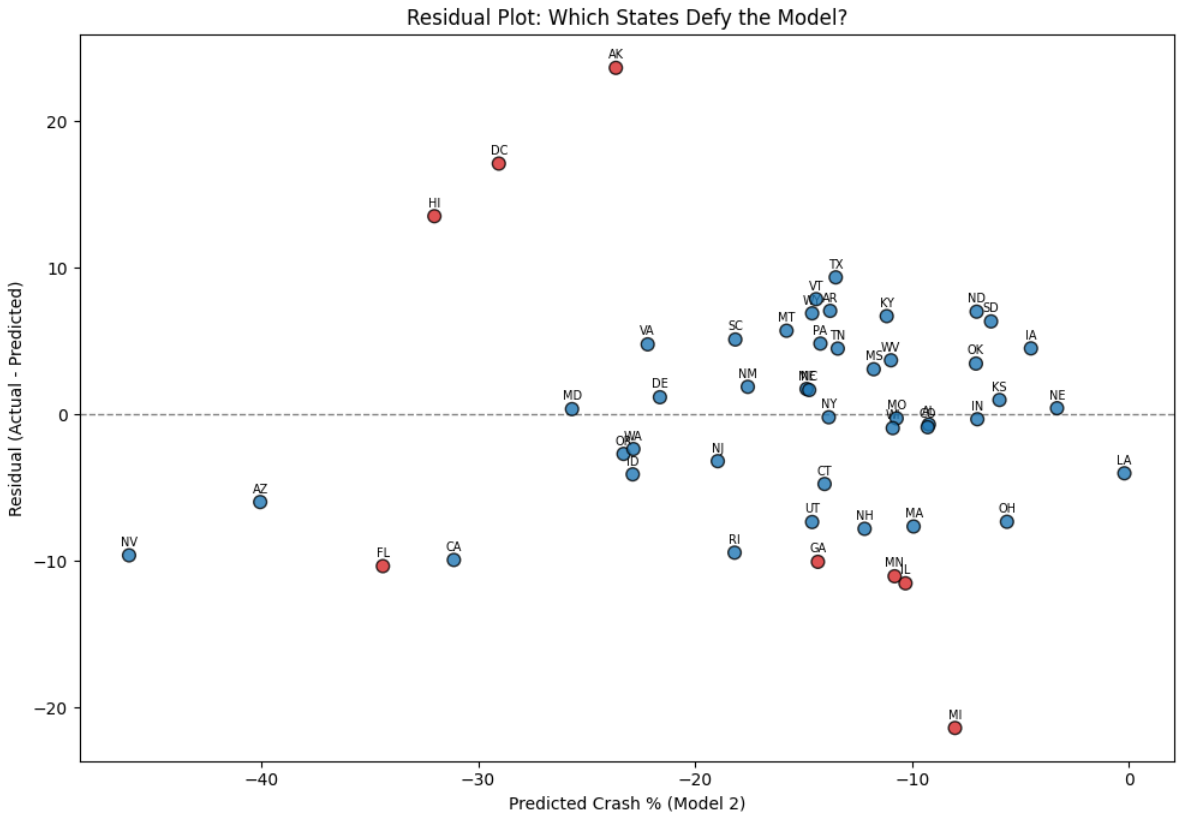
```
Observed R-squared: 0.4672
Permutation p-value: 0.0000
```



The observed R-squared is far above the null distribution, confirming that the relationship between pre-crisis appreciation and crash severity is not an artifact of chance. The bubble-crash link is real.

Which states defy the model?

The regression captures the general pattern, but some states crash much harder or softer than the multiple regression predicts. These outliers show what the model cannot capture: idiosyncratic economic structure that four pre-crisis indicators do not reflect.



Can an LLM agent outperform a regression?

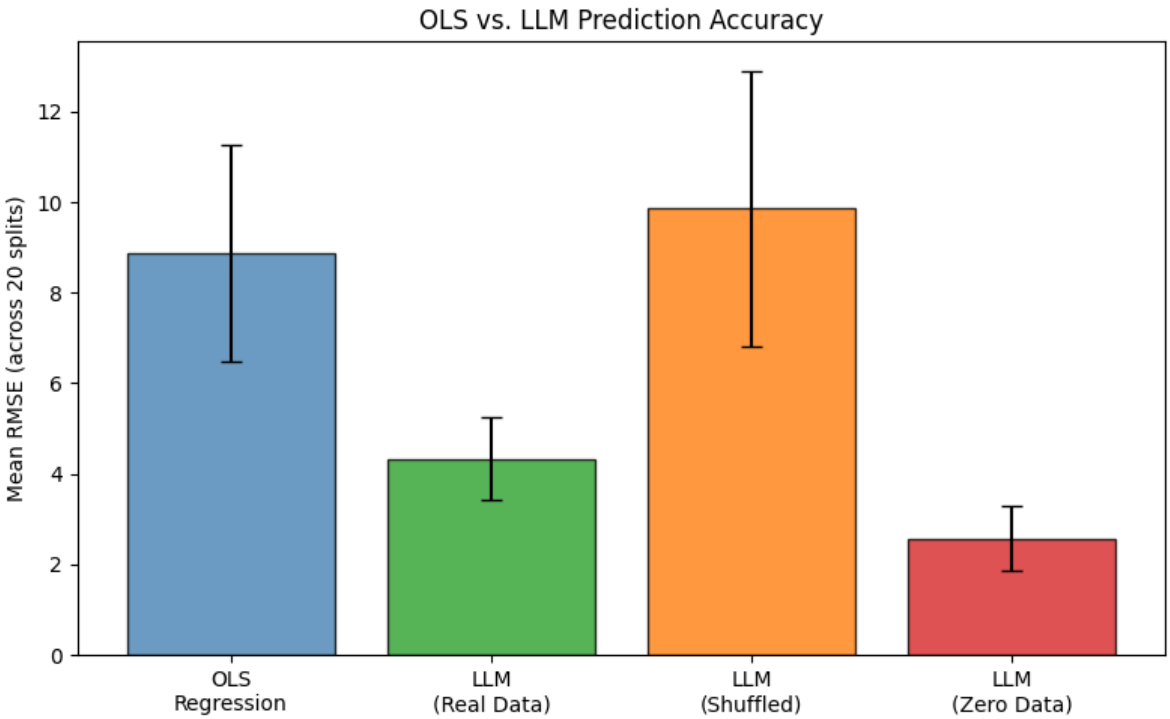
OLS has perfect computation but zero domain knowledge: it does not know that Arizona's 80% appreciation in a desert housing market differs from North Dakota's 20% in an oil economy. LLMs invert this, encoding economic knowledge from pre-training but expressing it through inference rather than exact optimization.

In Bayesian terms, OLS computes a maximum likelihood estimate in a four-dimensional linear space with no prior. An LLM holds a prior $p(\text{crash} \mid \text{state identity})$ from pre-training that encodes latent economic structure: some identifiable (industry composition, supply elasticity, subprime concentration), others unobservable and black-box. By reading the 40-state training table, the model constructs a likelihood $p(\text{features} \mid \text{crash})$ during the forward pass. No

statistics are pre-computed; the model extracts this structure from the raw data. Its prediction is a posterior:

$$p(\text{crash} \mid \text{features}, \text{state identity}, \text{context}) \propto p(\text{features} \mid \text{crash}) \cdot p(\text{crash} \mid \text{state identity})$$

We test four conditions: **Predict** (pure reasoning with real training data), **Predict + Tools** (Python with numpy/pandas, regression libraries banned), **RLM** (iterative multi-turn with Python REPL), and **Zero-data** (model receives only state names, no features or training examples). The zero-data condition isolates the prior term by removing all in-context data. A **shuffled** condition (training labels randomly reassigned across states) tests prior-likelihood conflict. Full methodology: llm_experiment.ipynb.




```
Mean RMSE - OLS:            8.88 (+/- 2.39)
Mean RMSE - LLM (real):     4.34 (+/- 0.91)
Mean RMSE - LLM (shuffled): 9.86 (+/- 3.04)
Mean RMSE - LLM (zero):     2.58 (+/- 0.72)
```

The zero-data condition changes the picture. Given only state names (no features, no training data, no role description), Claude Opus 4.6 achieves mean RMSE 2.58 (+/- 0.70), outperforming every other condition. The model has memorized state-level housing crash severity from pre-training. State-level FHFA HPI declines are among the most analyzed statistics in

modern economics, widely published in academic papers, government reports, and post-crisis analysis.

The four conditions form a hierarchy:

Condition	Mean RMSE	What it measures
Zero data	2.58	Pure recall from pre-training
Real data	4.34	Recall degraded by incomplete features
OLS	8.88	Features only, no prior
Shuffled data	9.86	Prior fought by wrong labels

Providing real training data (4 features, 40 states) degrades predictions from 2.58 to 4.34. The four features explain 58% of crash variance (see Model 2 above), meaning 42% of the information the model's prior already encodes cannot be expressed through the feature set. The likelihood term introduces a 4-dimensional bottleneck that partially overrides a stronger prior. This is a concrete instance of a Bayesian principle: when the prior is very informative and the likelihood is weak, the posterior is worse than the prior alone.

The shuffled condition (RMSE 9.86) shows the model does not ignore in-context data. Wrong labels pull it off its prior, degrading performance below OLS. The model incorporates training examples even when they are adversarial, which is why shuffling makes things worse rather than having no effect.

Method	Model	Split 0 RMSE
Predict	Claude Opus 4.6	3.16
Predict	GPT 5.4 Heavy	3.28
Predict	GPT 5.4 Pro	3.84
Predict	Gemini 3.1 Pro	3.87
Predict + Tools	GPT 5.4 Heavy	4.11
Predict	Gemini 3 Flash	4.21
RLM	GPT 5.4 Mini	4.71
OLS Regression	statsmodels	5.45

The latent space hypothesis

The leaderboard is ordered by model strength, not harness sophistication. Tools collapse the model's representational advantage. GPT 5.4 Heavy scores 3.28 without tools and 4.11 with them. With tools, it computed Washington's KNN average at -17.89. Without tools, it reasoned about the tech economy cushion and predicted -21.50, closer to the actual -25.19. Tools reduce the model's similarity function from its full latent space to the four provided columns.

Illinois provides direct evidence of latent variables at work. Every model except Opus under-predicts IL by 6+ points. The four-feature profile says "moderate Midwest state," but Opus reasons about subprime exposure on the south side, a variable absent from the dataset, and predicts within 2.81 points of the actual -21.81. GPT 5.4 Heavy misses by 6.41; Pro by 7.31.

Whether this latent knowledge comes from memorized crash statistics or from structured economic reasoning cannot be determined for this task. The 2008 housing crash is too well-documented to isolate the two mechanisms. Testing the predictive primitive hypothesis cleanly requires a target variable the model has not encountered during pre-training.

Conclusions

Pre-crisis appreciation alone explains 47% of cross-state crash variance ($R^2 = 0.47$, $p < 0.0001$). Adding fundamentals raises R^2 to 0.58 (F-test $p = 0.011$), with population growth as the strongest additional predictor ($p = 0.005$). The residuals show what four indicators miss: Michigan's auto collapse, Alaska/DC/Hawaii's insulation.

LLM experiment: memorization and prior-likelihood interaction

Claude Opus 4.6 achieves mean RMSE 4.34 across 20 splits versus OLS's 8.88. We initially interpreted this as evidence that LLMs could outperform statistical methods through latent economic knowledge. However, a zero-data condition (model receives only state names, no features or training data) achieves RMSE 2.58, revealing that the model has memorized state-level crash severity from pre-training.

This discovery reframes the experiment. The question "can an LLM outperform OLS?" is trivially answered when the model has memorized the target variable. The more interesting finding is what happens when memorized knowledge meets incomplete data. Four features explaining 58% of variance degrade the model's predictions from 2.58 to 4.34: the in-context data partially overrides a stronger prior with a weaker signal. Shuffled labels degrade further to 9.86, confirming the model incorporates training data even when adversarial. This is a Bayesian result:

$$p(\text{crash} \mid \text{features}, \text{state}, \text{context}) \propto \underbrace{p(\text{features} \mid \text{crash})}_{\substack{\text{likelihood from} \\ \text{training data}}} \cdot \underbrace{p(\text{crash} \mid \text{state})}_{\text{pre-training prior}}$$

When the prior is near-perfect (zero-data RMSE 2.58) and the likelihood is incomplete ($R^2 = 0.58$), the posterior is worse than the prior alone. This principle generalizes beyond our specific task: for any well-documented outcome, providing an LLM with incomplete features may hurt more than help.

Delivery mechanism findings

Regardless of memorization, the experiment reveals how different interfaces extract knowledge from the model. Pure reasoning outperforms tool-assisted prediction in every comparison. GPT 5.4 Heavy scores 3.28 without tools and 4.11 with them. Tools anchor the model on explicit features, collapsing its representational space from the full latent dimensionality to the four provided columns. Extended thinking also hurts: GPT 5.4 Pro's 28 minutes of deliberation (3.84) underperforms Heavy's 4 minutes (3.28). These findings transfer to settings where the model's knowledge comes from reasoning rather than recall.

Limitations

The predictive primitive hypothesis, that LLMs can function as frozen-weight prediction functions using latent structure from pre-training, remains plausible but unproven. The 2008 housing crash is too well-documented to disentangle memorization from generalized economic reasoning. A clean test requires a target variable absent from the training corpus: a novel economic indicator, a future event, or a synthetic outcome constructed from real features. This is a limitation of the current experiment, not a refutation of the hypothesis.

Follow-up: tool-computation deferral and clean-context review

Setup

Two-pass architecture: a generator model produces predictions in one context, then an Opus 4.6 reviewer evaluates those predictions in a clean context with no access to the generator's intermediate reasoning or tool calls. Tests whether the observed tools tax is correctable by stripping the computation-mode trajectory.

2x2 on split 0 (same 11 held-out states used throughout):

- Source: zero-data recall (Opus 4.6, state names only) vs tool-assisted prediction (GPT 5.4 Heavy with computation tools).
- Review: original predictions returned by the generator vs Opus 4.6 review over those same predictions using `prompts/review_predictions_minimal.md`, with no tools and no data beyond the prediction list.

Results (split 0)

Condition	RMSE
Zero-data (original)	2.48

Condition	RMSE
Zero-data + review	2.16
Tools (GPT 5.4 Heavy, original)	4.11
Tools + review	1.98

Tools + review is the lowest RMSE observed across every harness, model, and condition tested in this project. It beats pure zero-data recall (2.16) and it beats tools alone (4.11) by roughly 2x. The review pass recovers 76% of the tools-pass error (4.11 → 1.98).

State-by-state: tools vs tools + review

State	Actual	Tools	T+Rev	Tools err	T+Rev err	Action
WA	-25.19	-19.20	-24.00	+5.99	+1.19	helped
UT	-21.93	-15.80	-20.00	+6.13	+1.93	helped
IL	-21.81	-14.20	-25.00	+7.61	-3.19	helped
NM	-15.68	-14.80	-18.00	+0.88	-2.32	hurt
NY	-14.02	-17.80	-15.00	-3.78	-0.98	helped
NC	-13.06	-13.10	-13.10	-0.04	-0.04	neutral
MO	-10.97	-6.60	-9.00	+4.37	+1.97	helped
IN	-7.30	-7.10	-10.00	+0.20	-2.70	hurt
AR	-6.70	-8.40	-7.00	-1.70	-0.30	helped
LA	-4.21	-7.20	-5.50	-2.99	-1.29	helped
SD	0.00	-2.90	-2.90	-2.90	-2.90	neutral

Counts: helped 7/11, hurt 2/11, neutral 2/11. The review moved 9/11 predictions toward the Opus zero-data prior; the two unchanged states (NC, SD) were already close to the prior. Mean absolute adjustment on helped states: 4.01. On hurt states: 3.05.

The two hurt states (NM, IN) are the cases where the tools-pass prediction was already the closest to ground truth ($|\text{err}| < 1.0$). The review pushed them past ground truth because it had no signal that those specific predictions were already correct, only the generic imperative to second-guess the computation. The four largest review wins (WA, UT, IL, MO) correspond to states with initial tools-pass error above 4.0, in every case a state with strong public record for the 2008 crash (tech-hub bubble, subprime exposure, Midwest deindustrialization).

Interpretation: exposure bias at the reasoning level

Claim: in-context tool results function as teacher forcing at the reasoning level. When a tool output appears in the generation trajectory, downstream reasoning conditions on that output

as if it were ground truth. The instruction "use domain reasoning" does not override this because the tool output occupies a structurally privileged position in the context relative to an abstract instruction.

This is the structural analogue of exposure bias in autoregressive sequence models. In training, the model is conditioned on ground-truth prefixes at every step (teacher forcing). At inference, the model must condition on its own samples, and errors compound because it was never trained to recover from its own mistakes. The reasoning-level analogue: recent instruction-tuned models are post-trained heavily on retrieval-augmented and tool-use trajectories where the provided context is correct. At inference, when a subtly wrong but plausible computation enters the context, the model has no trained reflex to reject it.

The clean-context review recovers for the same reason a fresh forward pass can produce a different completion: the reviewer has no trajectory to be anchored by. It is evaluating prediction magnitudes against its own prior alone. Its prior is the Opus zero-data prior, which is known to be strong (RMSE 2.48 without review, 2.16 with self-review).

This also explains the hybrid effect where tools + review (1.98) beats zero-data alone (2.16, 2.48). The tools pass supplies a candidate set generated from 40 training states plus domain reasoning plus explicit computation. The reviewer shrinks that candidate set toward the prior. Prior-as-regularizer over a computation-refined candidate set outperforms prior alone. The reason this is not redundant with zero-data+review is that the candidate set contains state-specific signal that the zero-data pass has no access to (the reviewer is working from a tools-enriched base, not from raw pre-training recall).

Deferral mechanism

Summary claim: for instruction-adhering models, in-context grounding (RAG results, tool outputs, explicit computation traces) functions as an assumed-correct update to the prior rather than a hypothesis to be evaluated. The model's instruction-following tendency extends to treating context-provided computations as authoritative.

The failure is not the arithmetic. GPT 5.4 Heavy's KNN averages are computed correctly. The failure is the absence of a challenge step: the model does not ask "does my domain prior agree with this number?" before emitting it as the final answer. The review pass is exactly that challenge step, and because the reviewer has no trajectory anchoring it, it defaults to evaluating predictions against its own prior.

Direct evidence: the three largest tools-pass errors (UT, IL, WA) correspond to states with strong publicly documented crash severity. These are not states where GPT 5.4 Heavy lacks a prior. It had the prior and declined to use it to overrule the KNN result. The same domain prior, accessed in a clean-context review, corrected every one.

Cascade hypothesis (not yet proven, direct tests described)

Claim: once a tools-enabled model accepts one subtly-wrong computation as ground truth, the threshold for accepting the next subtly-wrong computation drops. Error compounds across the prediction sequence.

Mechanism: each accepted tool output reinforces the "tool output is the answer" generation pattern within the running context. By mid-sequence, the model is translating tool outputs into predictions rather than evaluating them. The first accepted wrong computation raises the prior probability of the pattern "accept computation" for the next state.

Suggestive evidence from split 0:

- The tools-pass errors are consistently one-sided for states where GPT 5.4 Heavy has strong domain knowledge (UT, IL, WA, MO all under-predict crash severity by 4 to 8 points). If the model were evaluating each prediction independently against its prior, we would expect some of these under-predictions to be corrected by the prior; none are.
- All four under-predictions are corrected during clean-context review, to within 1 to 3 points of ground truth, using the same underlying prior.

This is consistent with cascade but does not rule out a simpler explanation: that tools-pass predictions are uniformly biased toward the KNN-neighbor mean regardless of position in the sequence. The two hypotheses are distinguishable with the existing trace data. Concrete tests:

1. Residual autocorrelation within split. If the first wrong prediction raises the acceptance threshold for subsequent ones, within-split residuals should show positive autocorrelation in absolute error. Shuffle the prediction order in a re-run and test whether the autocorrelation persists under the new ordering.
2. Position regression. Regress $|\mathrm{err}_i|$ on prediction index i within split. Positive coefficient supports cascade.
3. Single-state-per-prompt ablation. Run 11 independent tools prompts, one state per prompt, and compare aggregate RMSE to the joint 11-state prompt. If per-state is substantially better, cascade is supported; if equal, the bias is uniform and cascade is unsupported.
4. Ordering perturbation. Shuffle the order the held-out states appear in the prompt and check whether high-error states cluster by position or by identity.

Ablations 1, 2, and 4 reuse the existing `results/cv_traces/` data. Ablation 3 requires one new batch of 11 independent runs.

Related work (online search, 2024-2026)

- Xie et al. 2024, "Adaptive Chameleon or Stubborn Sloth: Revealing the Behavior of LLMs in Knowledge Conflicts." Characterizes parametric-vs-contextual knowledge conflict. Finds that models over-defer to context, including when context contradicts stronger parametric knowledge. Does not test subtly wrong context (only obvious mismatches) and does not test post-hoc recovery via clean-context review.
- Madaan et al. 2023, "Self-Refine: Iterative Refinement with Self-Feedback." Same-model self-critique in the same context. Gains are modest and sometimes negative, because the critique step shares the anchoring context. Our clean-context review is structurally different and shows much larger recovery (76% of tools-pass error).
- Shinn et al. 2023, "Reflexion: Language Agents with Verbal Reinforcement Learning." Iterative reflection across trials; also in-context. Similar structural limitation to Self-Refine.
- Paul et al. 2024, "REFINER: Reasoning Feedback on Intermediate Representations." Separates generator and critic models for reasoning refinement. Closer to our setup, but the focus is multi-step math reasoning rather than context-poisoned prediction.
- Ranzato et al. 2016, "Sequence Level Training with Recurrent Neural Networks," and the exposure-bias literature that follows. Well-characterized at the token level. The reasoning-level analogue is, as far as we can find, not explicitly named in this literature.
- TRICE (2024), tool-calling feedback via RL. Trains the model to self-correct mid-trajectory; does not study the post-hoc clean-context recovery mechanism.

Novelty assessment: the component pieces (context-vs-prior conflict, separate-critic refinement, exposure bias) are each well-researched. The specific configuration below does not appear to have been isolated in prior work:

- The context pathology is subtly-wrong-but-convincing in-context computation (not hallucinated retrieval, not adversarial injection, not obviously contradictory retrieval). The computation is arithmetically correct; it is the model's uncritical acceptance of the computation's implication that is the bug.
- Recovery is via clean-context review by a model of equivalent or stronger prior, with no access to the original trajectory.
- The hybrid effect: the two-pass pipeline beats both the tool-assisted run and the pure-recall run separately. The first pass is not discarded; it is a candidate set the reviewer shrinks toward the prior.

Predictions the framing makes

If the exposure-bias-at-reasoning framing is correct:

- The tools tax should scale with prior strength. Tasks where the model has a strong memorized prior (our housing crash task) show a large tax. Tasks where the prior is weak

and context is the best signal (company-specific forecasting, novel-domain estimation) should show no tax or negative tax.

- Review recovery requires the reviewer's prior to be at least as strong as the generator's. A weaker reviewer should recover less. A reviewer with a different prior should shrink toward that different prior.
- Tools + review residuals should correlate more strongly with zero-data residuals than with OLS residuals on the same held-out states. The review is pulling predictions toward the prior, not toward the regression.
- Residual autocorrelation within a split (cascade test) should be stronger in the tools condition than in the zero-data condition.

Each prediction is a testable follow-up. None require new data collection beyond what is already in `results/cv_traces/` and `results/`.

Artifacts

- Zero-data split 0 generator: `results/cv_traces/split_00_zero.md` (Opus 4.6, max reasoning).
- Tools split 0 generator: `results/gpt54-heavy-tools-split0.md` (GPT 5.4 Heavy).
- Review prompt: `prompts/review_predictions_minimal.md`.
- Review runs for both conditions were executed as one-shot Opus 4.6 calls with `--effort max` over the prediction lists from the two generator traces; the prediction deltas are tabulated above.

Reflection

To be completed.

Appendix

LLM experiment methodology and per-model analysis: llm_experiment.ipynb

Source code and data: github.com/ethanaggor/sds1730-final-project